Paxos and Raft Consensus Algorithms Explained

ID: DOC-NET-803 | Category: Distributed Systems and Networking | Version: 2.0

1. The Distributed Consensus Problem

In a distributed system, individual nodes often fail, and networks experience partitions. To maintain a consistent state across a cluster of independent machines (like a distributed database or key-value store), the cluster must agree on a single value or a sequence of operations. This agreement must remain mathematically proven and uncorrupted even if some nodes crash or messages are delayed. This is the problem of Distributed Consensus.

2. Paxos: The Theoretical Pioneer

Introduced by Leslie Lamport, Paxos is the foundational algorithm for distributed consensus. It defines roles: Proposers (advocate a value), Acceptors (vote on values), and Learners (record the outcome). Paxos operates in two phases: a Prepare phase to establish a sequence number, and an Accept phase to vote on the value. While theoretically elegant and strictly mathematically proven to guarantee safety, pure Paxos is notoriously difficult to understand and even harder to implement in practical software architectures. Implementations often diverge heavily into complex variants (Multi-Paxos), leading to fragmented, bug-prone architectures.

3. Raft: Designed for Understandability

Raft was specifically created as a reaction to the opacity of Paxos. It provides the same exact fault-tolerance and performance but decomposes the consensus problem into three distinct, understandable sub-problems:

- **Leader Election:** Unlike Paxos, Raft enforces a strong leader paradigm. Nodes operate as either a Leader, Follower, or Candidate. If a follower stops receiving heartbeats, it becomes a candidate, increments a term counter, and requests votes. A candidate that secures a majority vote becomes the new leader.
- **Log Replication:** The leader acts as the sole ingress for client requests. It appends the request to its local log and sends AppendEntries RPCs to followers. Only when a majority of followers acknowledge the write does the leader commit the entry and apply it to its state machine.
- **Safety constraints:** Raft guarantees that if any state machine applies a log entry at a specific index, no other server will ever apply a different log entry for that same index. The election process inherently restricts which servers can be elected leader (only those possessing all committed entries).

4. Practical Applications

Consensus algorithms do not manage standard bulk application data due to their latency overhead. Instead, they act as the coordination core of distributed networks. Apache ZooKeeper (using its ZAB protocol, similar to Raft) manages configuration and coordination for Hadoop and Kafka. HashiCorp Consul and Kubernetes' backing store, **etcd**, are built directly on top of the Raft algorithm to guarantee the strict consistency of critical infrastructure metadata.